



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Voice2Query: AI-Powered Speech-to-SQL for Interactive Database Exploration

Fundamentals of Programming and Data Management

Vincenzo Moscato

Keywords: WrenAI, Speechmatics, WhisperAI, Speech-to-Text, SQL DB,
Natural Language Processing, Ollama

Giovanni Fierro, Marco Apuzzo, Stefania Rosciano, Belal
Hmeidat

June 17, 2026

Contents

1	Introduction	1
1.1	General System Architecture	1
1.1.1	FastAPI Backend	2
1.1.2	WrenAI Container (WrenAI Engine & Configuration Console / WrenAI UI)	2
1.1.3	DBMS Container (PostgreSQL)	3
1.1.4	User Interface (Streamlit)	3
2	Whisper AI v. Speechmatics	3
2.1	Transcription Accuracy	3
2.2	Latency	3
2.3	Hardware and Software Requirements	4
2.4	Ease of Integration	4
2.5	Multilingual Support	4
2.6	Speaker Diarization	4
2.7	Comparison Table	4
2.8	Conclusion	5
3	Text to SQL with WrenAI	5
3.1	Underlying Model	5
3.2	Database Compatibility	5
3.3	Issues and Limitations	6

List of Figures

1	Database Schema	1
2	System Architecture Diagram	2

List of Tables

1	Comparison of Whisper AI and Speechmatics	4
---	---	---

1 Introduction

The **Voice2Query** project focuses on the design and implementation of a Speech-to-SQL pipeline, developed to allow users to interact with a relational database using spoken natural language.

The main goal is to break down the technical and syntactic barriers related to SQL knowledge for non-expert users, providing an intuitive and immediate access interface. The system captures voice input through a dedicated graphical interface, transcribes it into text, translates the text into a correct SQL query consistent with the database, and finally displays the results on an interactive dashboard.

To start the project, we needed to create a database to work with. We created a PostgreSQL database containing 3 tables with information about games, gamers, and reviews they gave to games. Figure 1 illustrates the database schema, showing the relationships between the tables and the fields contained in each table. The database was populated with synthetic data.

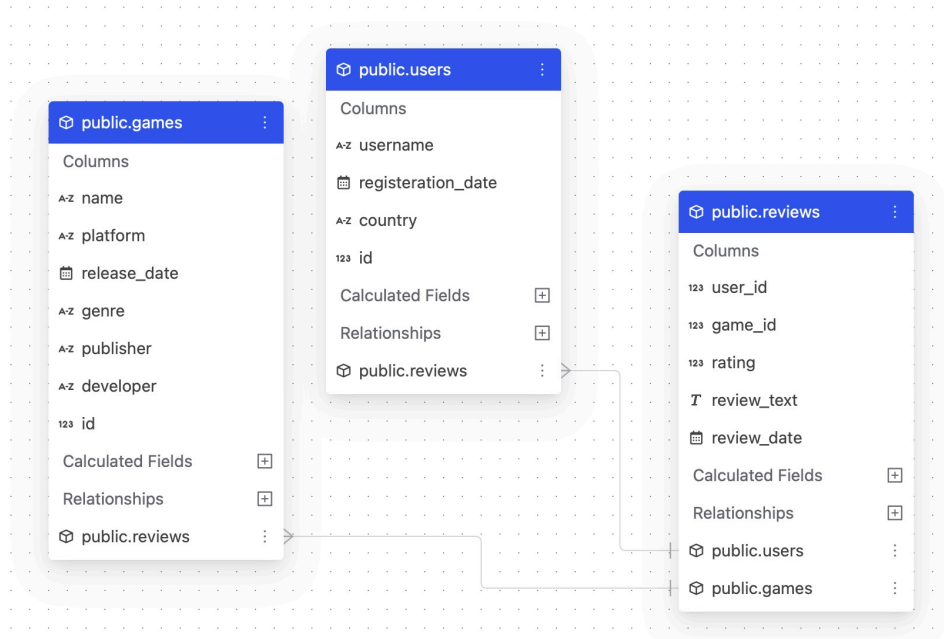


Figure 1: Database Schema

1.1 General System Architecture

The system is based on a cascaded pipeline architecture, widely validated in scientific literature as a modular alternative to pure end-to-end models. To ensure service isolation, environment reproducibility, and infrastructure portability, the core of the pipeline was engineered through a hybrid Docker containerized and non containerized configuration. The network topology of the project involves a clear separation between core services and the client interface and is composed of 4 main components: the user interface, the FastAPI backend, the WrenAI container, and the DBMS container. Each component is designed to perform specific functions while maintaining clear communication channels with the others, ensuring

a seamless user experience. Figure 2 illustrates the overall architecture of the system, highlighting the interactions between the different components and the flow of data.

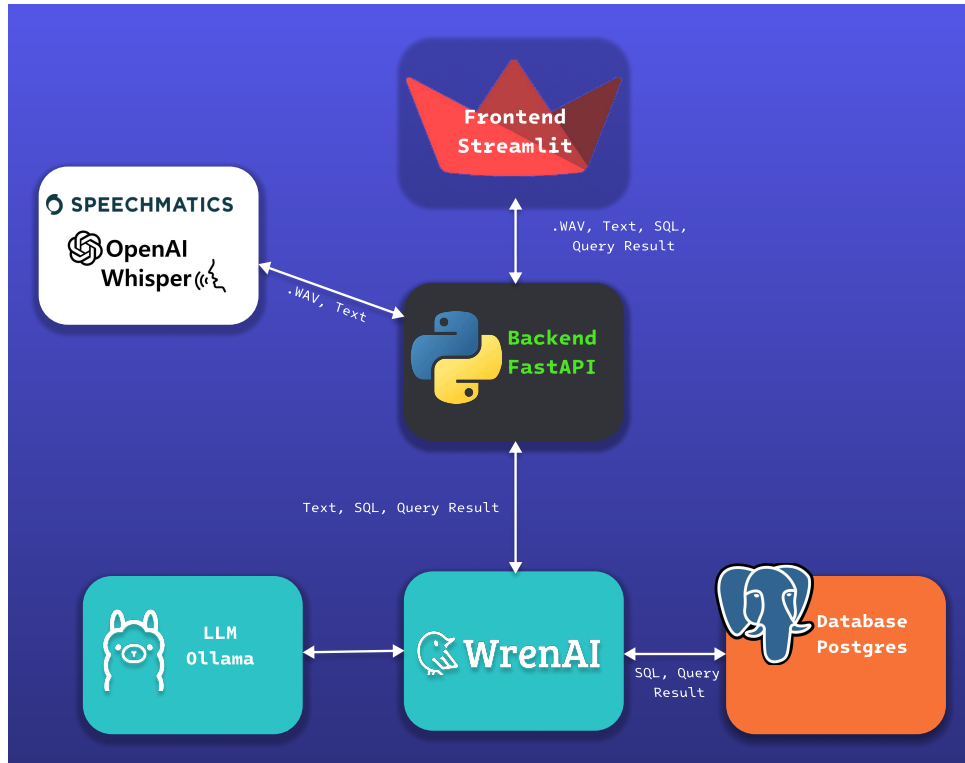


Figure 2: System Architecture Diagram

The following describes each component in more detail:

1.1.1 FastAPI Backend

The FastAPI backend serves as the central hub for processing user requests. It separates the user interface from WrenAI and the database, ensuring that the system remains modular and maintainable. All communications from and to the backend are done with RestAPI. The backend is responsible for receiving audio files from the client in the .wav format, forwarding them to the speech-to-text module: Speechmatics or Whisper, then sending the transcribed text to WrenAI for SQL query generation, executing the generated SQL query on the database, and finally sending the results back to the client for visualization.

1.1.2 WrenAI Container (WrenAI Engine & Configuration Console / WrenAI UI)

This module runs the WrenAI engine. It houses both the logical back-end (which communicates with remote LLMs and the database) and its WrenAI UI (the Configuration Console). The latter is used exclusively during the development phase for establishing connection with the database, indexing, and semantically mapping the database metadata.

1.1.3 DBMS Container (PostgreSQL)

Hosts the active instance of the relational database. It is accessed by the WrenAI service and runs within an isolated Docker container, remaining shielded from unauthorized external access.

1.1.4 User Interface (Streamlit)

The main client application runs natively on the host machine. This choice is driven by the need to interface directly with local hardware peripherals for audio capture (microphone) and to maximize the responsiveness of the graphical dashboard during chart rendering. Streamlit communicates with the backend through REST API calls, sending audio data, transcription, and receiving queries, and results for visualization.

Ollama

To ensure the computational sustainability of the project, it was decided to exclude Ollama from the local Docker stack. The computationally intensive processes related to SQL code generation are delegated to enterprise-class Large Language Models accessible via external cloud APIs. This architectural design decision avoids overloading the local RAM and GPU VRAM, allowing the entire stack to run smoothly even on standard hardware architectures (such as thin clients or laptops without dedicated GPUs).

2 Whisper AI v. Speechmatics

2.1 Transcription Accuracy

We tested both models with sample text we read and recorded in clear English, and both models performed well with few errors. The test shows WhisperAI performing better on both WER and CER speech-to-text metrics, with approximately a 45% improvement in WER. However, it should be noted that our tests aren't comprehensive since they don't evaluate the models with different languages, dialects, or noise levels.

2.2 Latency

Another aspect to consider when considering the two solutions is the latency and live transcribing availability. Speechmatics supports real-time transcription, whereas WhisperAI works with batch processing of audio, which increases the time needed to transcribe in case of large audio files.

2.3 Hardware and Software Requirements

The two tools have different requirements as Whisper AI gets installed locally and runs on the device or a private server. On the other hand, Speechmatics is a cloud-based service that saves the user a lot of resources.

2.4 Ease of Integration

Speechmatics, being a cloud-based solution, requires only API integration. WhisperAI, on the other hand, runs locally, which is good for offline workflow but makes any dependent package very large in size. Another option is to send requests to a dedicated private server running WhiserAI, but this requires maintaining additional servers, which in turn makes the integration process harder. However, Speechmatics might turn out to be more expensive in the long run than WhisperAI, as it is sold as a service, whereas WhisperAI is open source and free to utilize.

2.5 Multilingual Support

Speechmatics supports 50+ languages, whereas WhisperAI supports 99 languages. This is probably due to the nature of the open nature of the WhisperAI project whereas Speechmatics is a proprietary service. However, the performance of both models on different languages requires futher testing.

2.6 Speaker Diarization

WhisperAI does not support speaker diarization natively. Instead, a community project called WhisperX adds this capability, utilizing the leading open-source solution provided by Pyannote. The open source version of Paynnote performs with an average Diarization Error Rate (DER) of 11.2% [1],so WhisperX is expected to perform similarly. On the other hand, Speechmatics supports speaker diarization natively, which is a significant advantage for users who require this feature. No public data is available on the performance of Speechmatics diarization, but it is expected to perform better than WhisperX since it is a paid service and not an open-source project.

2.7 Comparison Table

Feature	Whisper AI	Speechmatics
Transcription Accuracy	Very High	High
Live Transcription	Batches Only	Yes. Also supports batches
Hardware/Software Requirements	Local Installation	Cloud-Based
Ease of Integration	Moderate	Easy
Multilingual Support	99+ Languages	50+ Languages
Speaker Diarization	No (via WhisperX)	Yes

Table 1: Comparison of Whisper AI and Speechmatics

2.8 Conclusion

In conclusion, both Whisper AI and Speechmatics have their own strengths and weaknesses. Whisper AI excels in transcription accuracy and multilingual support, while Speechmatics offers real-time transcription and native speaker diarization. The choice between the two will depend on the specific needs of the user, such as the importance of real-time transcription, the need for speaker diarization, and budget constraints. For users who prioritize accuracy and multilingual support, Whisper AI may be the better choice. However, for those who require real-time transcription and native speaker diarization, Speechmatics may be more suitable.

In this project, we opted to use both solutions as they are both compatible with our architecture. The user is provided with an option in the UI to choose between the two solutions.

3 Text to SQL with WrenAI

WrenAI is a powerful tool that allows for the translation of natural language queries into SQL code. It is designed to work with a variety of databases. The process of converting text to SQL with WrenAI involves several steps, including database indexing, semantic mapping, and query generation, all of which have been implemented in our project.

3.1 Underlying Model

We opted to use a legacy version Open Source of WrenAI that is free and runs locally on the machine to avoid WrenAI cloud service which costs money to use its tokens. This however requires coupling WrenAI with Ollama, which is a local Open Source LLM hosting solution. The WrenAI stack which includes WrenAI UI, engine, and service runs in a dedicated Docker container, while Ollama runs natively on the host machine to ensure optimal performance. The WrenAI engine is responsible for processing the transcribed text and generating the corresponding SQL queries, while the WrenAI UI is used for configuration and management of the WrenAI engine. The WrenAI service enables communication with the WrenAI engine using REST API calls, allowing for seamless integration with our FastAPI based backend.

3.2 Database Compatibility

WrenAI supports a wide range of databases, including PostgreSQL, MySQL, and SQLite. In our project, we used PostgreSQL as our database management system since it is widely used in the industry and offers robust features for handling complex queries.

3.3 Issues and Limitations

Using the legacy version of WrenAI means that the accuracy of the generated SQL queries may not be as high as desired. We saw this in our testing, where some of the generated SQL queries were not correct and required manual correction. Correction can be done by the user through our custom UI to the transcribed text. Furthermore, needing to use Ollama to run the LLM locally adds more latency and accuracy issues since the LLMs that can be run locally are not as powerful as the ones available on the cloud. Nonetheless, using the local version of WrenAI and Ollama allows us to avoid the costs associated with using the cloud-based WrenAI service, making it a more cost-effective solution for our project.

References

- [1] Benchmarking Diarization Models, arxiv.org/html/2509.26177v1. Accessed 11 May 2026.
- [2] Speechmatics, “Speechmatics Documentation,” 2024.